ISTA 130 – All About Functions

Purpose. Functions allow us to bundle multiple lines of code together, give that bundle of code a name, and to later execute that bundle in one line of code by calling it (using its name in our code). This makes it possible to:

- ***Decompose*** our large, complex problem into smaller, simpler problems that are easier to solve.
- Write clear, concise programs, i.e. programs that are short and understandable. As a part of this, good choice of function names helps anyone who is reading our code, including us, to understand how it works and what it does.
- Reuse code, saving us time and effort.
- Make our code more maintainable, i.e. easy to understand, fix, and modify (perhaps adding new functionality) later.
- ***Modularize*** our code – in other words breaking it down into simpler components that we can use to build our complicated finished programs. This bullet point encompasses the previous four.

Defining functions. First, some vocabulary (for any additional terms that you don't recognize or for expanded definitions, use the glossary provided in the Review module):

- ***Variable***: a name that refers to a value stored somewhere in the computer. The value associated with the name can change, i.e. vary; hence the term 'variable'.
- ***Literal***: An explicit value in our code.
- ***Parameter***: a special kind of variable that appears in the first line of a function definition and can be used throughout the function.
- ***Simple statement***: a single line of Python code.
- ***Statement***: a compound or simple statement. Often used as shorthand for simple statement.
- ***Compound statement***: a compound statement in Python consists of a header line that ends in a colon and a body. The body consists of one or more Python statements immediately following the header line, and is indented one level more than the header.

We want our functions to be flexible, which means we want them to be able execute the same sequence of statements using different starting values for the variables in those statements. Therefore, we need a way to get information into the functions. For the purposes of this class, we have three ways to get information into functions:

1. Get keyboard input from the user.
2. Read from a file on the computer's hard disk or SSD memory.
3. Pass it to the function when we call it using parameters.

We are interested in number 3 at this time. Let's look at some example code including a function definition:

```
1  import math
2
3  def angle(x1, y1, z1, x2, y2, z2):
4      '''
5      Return the angle between the real-valued vectors
6      u = [x1, y1, z1] and v = [x2, y2, z2] in degrees.
7      '''
8      dot_product = x1 * x2 + y1 * y2 + z1 * z2
9      length_u = math.sqrt(x1**2 + y1**2 + z1**2)
10     length_v = math.sqrt(x2**2 + y2**2 + z2**2)
11     angle = math.acos(dot_product / length_u / length_v)
12     return angle * 180 / math.pi
13
14 print(angle(1, 0, 0, 0, 0, 1))
```

Line 3 is the first line of the function definition, called the ***function signature/header***. The function signature consists of the keyword `def`, followed by the function name, followed parentheses. These parentheses optionally contain a sequence of parameters separated by commas. These parameters are variables in the function definition that act as placeholders. They have no values associated with them in the function definition. We associate values with the parameters when we call the function, for example on line 14. We ***call*** the function by using its name and passing one ***argument*** for each of the parameters in the function signature. Python then uses the arguments to set the starting values for the parameters and executes the body of the function (the ***body*** of the function is the code immediately beneath the signature and indented one level more than the signature). In this example, `x1` starts off with the value `1`, `y1` with the value `0`, etc. When the above code executes, Python prints `90.0` to the console (the vectors are perpendicular). We can call the function again, passing new starting values:

```
16 print(angle(1, 0, 0, 1, 0, 1))
```

This time, Python prints `45.0` because the function body executed with different starting values for the parameters.

Understanding how functions are defined and called is the hardest thing we do this semester. Here are some important points:

- function definition vs. function call: a function definition is a compound statement starting with the keyword `def`. A function call is an expression consisting of the function name followed by parentheses containing a sequence of zero or more arguments separated by commas.
- parameter vs. argument: parameters are in function definitions, arguments are in function calls. Parameters are variables, arguments may be variables, literals, or more complicated expressions.

We also need to get information out of functions. For the purposes of this class, we do this by printing to the console, writing to files, and with return statements. Return statements use the keyword `return` and tell Python what value the function should evaluate to.

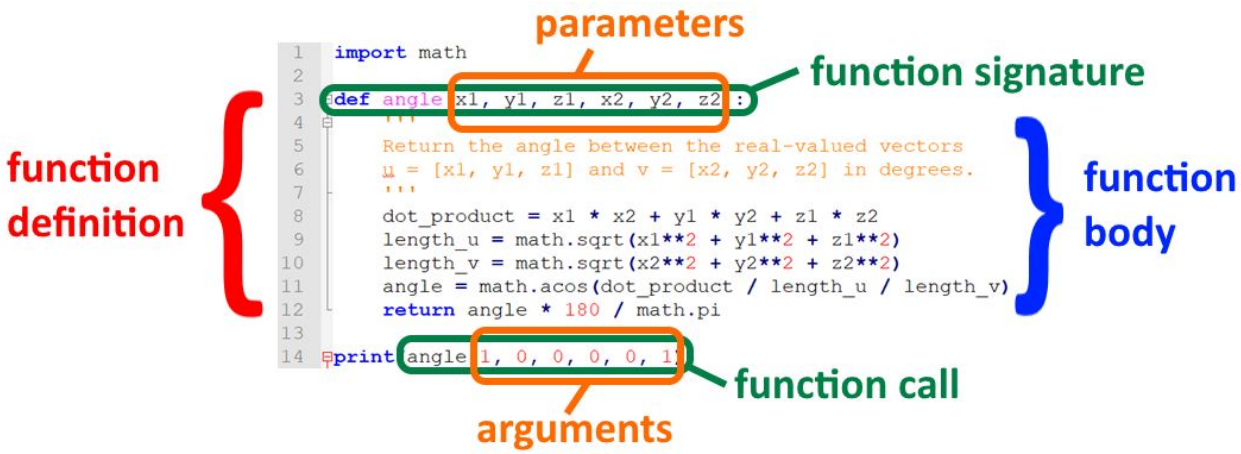
Test your understanding (answers on the last page). Refer to lines 1-14 in the code snippet above:

1. What line(s) contain function signatures?
2. What line(s) contain function calls?
3. What line(s) contain variables?
4. List all of the variables.
5. What line(s) contain function bodies?
6. What line(s) contain literals?
7. What line(s) contain parameters?
8. What line(s) contain arguments?
9. What line(s) contain assignment statements?
10. Are function calls expressions?
11. What line(s) contain return statements?
12. How many function calls are there on line 14?
13. What is the argument to the `print` function call on line 14?

Functions glossary.

- ***Argument***: The values between the parentheses in a function call are called arguments. These values may variables, literals, or more complicated expressions. Python uses them to set the initial values of their corresponding parameters in the function signature and then executes the body of the function definition. Multiple arguments in a function call are separated by commas.
- ***Function***: Code that is bundled together and given a name so that it may be executed in one line of code.
- ***Function body***: The code (including any docstring) immediately following the function signature that is indented one level more than the function signature. This is the code that is executed when the function is called. The function body must contain at least one statement.
- ***Function call***: An expression consisting of a function name followed by parentheses containing zero or more arguments separated by commas. When Python encounters a function call, it assigns the argument values to the corresponding parameters in the function definition and executes the body of the function definition.
- ***Function definition***: A compound statement consisting of a function signature and a function body.
- ***Function header***: Same thing as a function signature.
- ***Function signature***: The first line of a function definition. It begins with the keyword `def`, followed by the function name and parentheses containing zero or more parameters.
- ***Method***: A special kind of function called on an object using the dot operator. Example: the turtle method `forward`.
- ***Parameter***: A special kind of variable that appears in between the parentheses in a function signature (the first line of a function definition) and can be used throughout the function. It is a placeholder that has no initial value assigned to it in the function definition. It gets its initial value when other code calls the function.
- ***Return statement***: A statement in a function definition using the keyword `return` that tells Python what value the function should evaluate to when it is called.

Functions illustrated glossary.



Answers:

1. 3
2. 9-11, 14
3. 3, 8-12
4. `x1, y1, z1, x2, y2, z2, dot_product, length_u, length_v, angle, math.pi`
5. 4-12
6. 9, 10, 12, 14
7. 3, 8-10
8. 9-11, 14
9. 8-11
10. yes
11. 12
12. 2
13. a function call to the `angle` function